

# UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información

Inteligencia Artificial — IA26

## ENTRENADOR PERSONAL CON ESTIMACIÓN DE POSES EN TIEMPO REAL

*Descripción Técnica del Proyecto*

| Carné         | Nombre completo                  |
|---------------|----------------------------------|
| 9490-21-7     | Gerardo Antonio Ovando Hernandez |
| 9490-20-26029 | Elías Gregorio Marquirez         |
| 9490-20-26601 | Edwar Daniel Calderon Cinco      |
| 9490-20-10699 | Christian Aldair Rosales García  |
| 9490-20-6611  | Henry David Cabrera Virula       |
| 9490-19-11174 | Edgar Javier Nij Manchamé        |

**Fecha de entrega:** 22 de mayo de 2026

**URL de la aplicación:** <https://entrenador-personal-ia-qodm.vercel.app/>

**Repositorio:** <https://github.com/govandoh/entrenador-personal-ia.git>

## Contenido

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Resumen ejecutivo.....                             | 3  |
| 2. Introducción.....                                  | 3  |
| 3. Objetivos.....                                     | 4  |
| 3.1 Objetivo general.....                             | 4  |
| 3.2 Objetivos específicos.....                        | 4  |
| 4. Marco teórico.....                                 | 4  |
| 4.1 Estimación de poses humanas.....                  | 4  |
| 4.2 MediaPipe Pose Landmarker.....                    | 4  |
| 4.3 Aplicaciones web progresivas (PWA).....           | 5  |
| 5. Arquitectura del sistema.....                      | 5  |
| 5.1 Visión general.....                               | 5  |
| 5.2 Pipeline de detección.....                        | 6  |
| 5.3 Cálculo de ángulos articulares.....               | 6  |
| 5.4 Máquinas de estados y conteo de repeticiones..... | 6  |
| 5.5 Sistema de retroalimentación.....                 | 7  |
| 6. Ejercicios implementados.....                      | 7  |
| 6.1 Sentadilla (Squat).....                           | 7  |
| 6.2 Curl de bíceps (Bicep Curl).....                  | 8  |
| 6.3 Press de hombro (Shoulder Press).....             | 8  |
| 7. Stack tecnológico.....                             | 9  |
| 8. Decisiones de diseño relevantes.....               | 9  |
| 8.1 Decisión de servidor e infraestructura.....       | 9  |
| 8.2 Service Worker con estrategia diferenciada.....   | 9  |
| 8.3 Onboarding de cuatro pantallas.....               | 10 |
| 8.4 Delay de hardware al cambiar de cámara.....       | 10 |
| 9. Resultados.....                                    | 10 |
| 10. Limitaciones conocidas.....                       | 11 |
| 11. Conclusiones.....                                 | 11 |
| 12. Referencias.....                                  | 12 |

## 1. Resumen ejecutivo

Se desarrolló una aplicación web progresiva (PWA) que actúa como entrenador personal de ejercicios físicos utilizando visión por computadora en tiempo real. La aplicación accede a la cámara del dispositivo móvil del usuario, detecta su postura corporal mediante el modelo pre-entrenado MediaPipe Pose Landmarker de Google, y calcula ángulos articulares para determinar la calidad de ejecución de tres ejercicios: sentadilla, curl de bíceps y press de hombro. El sistema contabiliza repeticiones automáticamente, proporciona retroalimentación visual codificada por colores y emite instrucciones de voz en tiempo real, todo sin necesidad de conexión a un servidor externo ni de ningún costo económico.

El componente de inteligencia artificial del proyecto reside en la inferencia con un modelo de red neuronal profunda que estima la posición de 33 puntos corporales a partir de cada fotograma de video, combinado con máquinas de estados basadas en reglas angulares clínicamente fundamentadas para evaluar la calidad del movimiento.

## 2. Introducción

El sedentarismo y la falta de acceso a orientación profesional en el ejercicio físico son problemas de salud pública documentados en toda Latinoamérica. Las soluciones comerciales existentes — aplicaciones con entrenadores virtuales o dispositivos portátiles con sensores inerciales— presentan barreras económicas significativas para una gran parte de la población.

Este proyecto busca demostrar que la visión por computadora, combinada con los recursos gratuitos disponibles hoy en la web, puede democratizar el acceso a retroalimentación técnica de calidad durante el ejercicio. Un teléfono inteligente con navegador moderno es suficiente para ejecutar la inferencia de un modelo de estimación de poses sin necesidad de hardware especializado, conexión permanente a internet ni cuentas de pago.

Desde la perspectiva académica, el proyecto ilustra una aplicación concreta del área de análisis de imágenes dentro de la inteligencia artificial: el modelo de MediaPipe Pose es el producto de entrenamiento supervisado sobre millones de imágenes anotadas, y la aplicación consume ese modelo para resolver un problema de dominio real.

### 3. Objetivos

#### 3.1 Objetivo general

Desarrollar una aplicación web progresiva que utilice estimación de poses en tiempo real para guiar y evaluar la ejecución técnica de ejercicios físicos desde un dispositivo móvil, sin costo económico para el usuario ni infraestructura de servidor.

#### 3.2 Objetivos específicos

1. Integrar el modelo MediaPipe Pose Landmarker en un pipeline de procesamiento de video en tiempo real con latencia perceptualmente nula para el usuario.
2. Implementar máquinas de estados basadas en ángulos articulares para tres ejercicios: sentadilla, curl de bíceps y press de hombro.
3. Diseñar un sistema de retroalimentación multimodal (visual y auditivo) que comunique la calidad de ejecución sin interrumpir el flujo del ejercicio.
4. Empaquetar la aplicación como PWA instalable en dispositivos Android e iOS.
5. Publicar la aplicación en una URL pública de acceso gratuito.

### 4. Marco teórico

#### 4.1 Estimación de poses humanas

La estimación de poses humanas (human pose estimation) es una tarea de visión por computadora que consiste en localizar, en una imagen o fotograma de video, los puntos clave del esqueleto humano — articulaciones como hombros, codos, rodillas y caderas— para inferir la postura del cuerpo. Las redes neuronales convolucionales (CNN) entrenadas sobre grandes conjuntos de datos anotados han alcanzado precisión suficiente para ejecutarse en tiempo real, incluso en hardware de consumo, a partir de mediados de la década de 2010.

Los modelos actuales de detección de una sola persona (single-person) son especialmente eficientes porque evitan el costo computacional de la detección de múltiples personas, lo que los hace adecuados para aplicaciones de fitness donde el contexto garantiza un único sujeto en escena.

#### 4.2 MediaPipe Pose Landmarker

MediaPipe es una plataforma de soluciones de visión por computadora desarrollada por Google. El modelo Pose Landmarker detecta 33 puntos corporales en cada fotograma, cada uno con coordenadas normalizadas ( $x, y \in [0,1]$ ) relativas al tamaño de la imagen, coordenada de profundidad relativa ( $z$ ) y un indicador de visibilidad ( $v \in [0,1]$ ).

Este proyecto utiliza la variante Lite del modelo, optimizada para velocidad con una leve reducción de precisión respecto a la variante Full, lo que resulta adecuado para un entorno móvil donde la potencia de cómputo es limitada y la latencia es crítica para la experiencia de usuario.

La API utilizada es `@mediapipe/tasks-vision` (Tasks API, versión 0.10.35), la interfaz moderna y unificada de MediaPipe para JavaScript, que reemplaza a la API legacy y ofrece un modo de ejecución VIDEO diseñado específicamente para el patrón `requestAnimationFrame`.

### 4.3 Aplicaciones web progresivas (PWA)

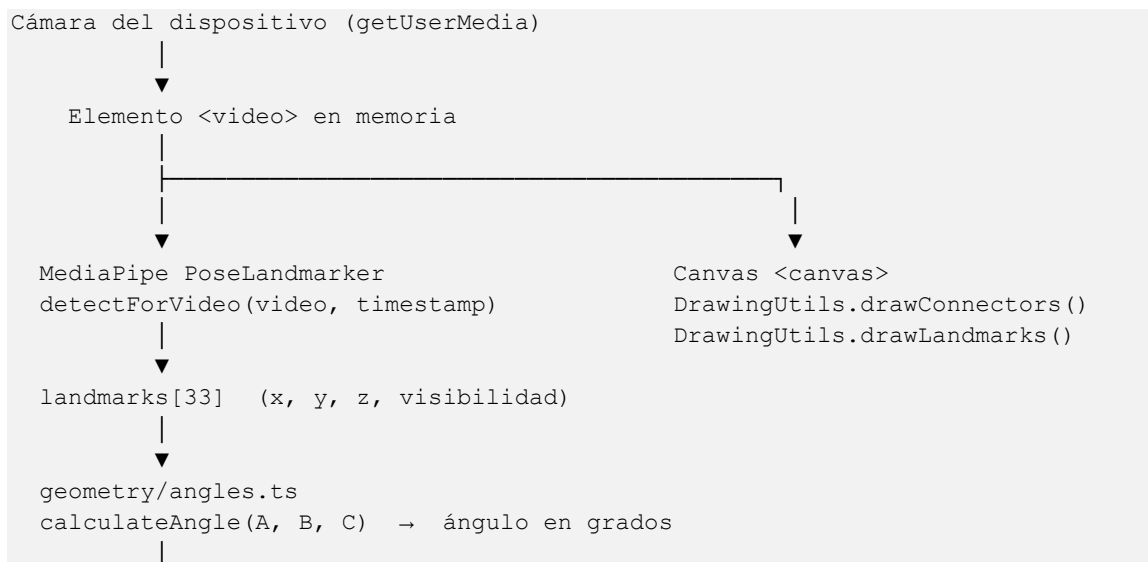
Una Aplicación Web Progresiva es una aplicación web que adopta un conjunto de tecnologías del navegador —principalmente el Service Worker y el Web App Manifest— para comportarse como una aplicación nativa instalable. Las PWA pueden ejecutarse sin conexión (o con conectividad limitada), aparecer en la pantalla de inicio del dispositivo como cualquier otra aplicación, y eliminar la barra de navegación del navegador para ofrecer una experiencia de pantalla completa.

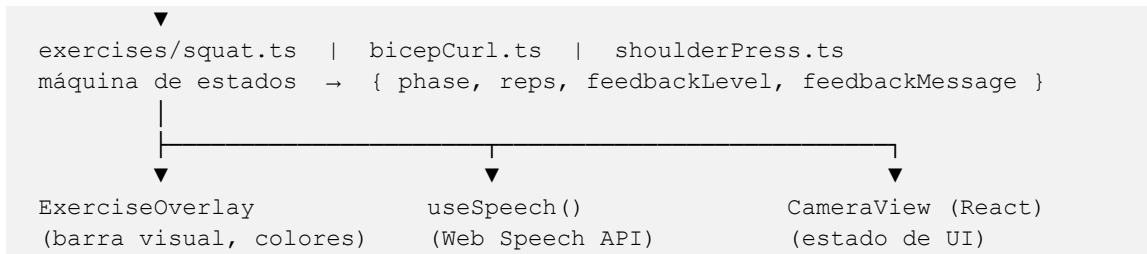
Para una aplicación de fitness como la desarrollada, el modelo PWA es ideal: el usuario instala la aplicación una sola vez y accede a ella directamente desde su pantalla de inicio sin fricciones de apertura de navegador.

## 5. Arquitectura del sistema

### 5.1 Visión general

La aplicación es completamente del lado del cliente (client-side only). No existe servidor de aplicación: toda la lógica de detección, cálculo y retroalimentación ocurre en el procesador del dispositivo del usuario. El diagrama de flujo de datos es el siguiente:





Este diseño garantiza que la aplicación funcione sin conexión a internet una vez que el modelo de MediaPipe ha sido descargado y cacheado por el Service Worker y el caché HTTP del navegador.

## 5.2 Pipeline de detección

En cada iteración del bucle de animación (típicamente 30-60 fotogramas por segundo), CameraView ejecuta la función `detectAndDraw()` pasando el fotograma actual del elemento `<video>` y una marca de tiempo en milisegundos. MediaPipe ejecuta la inferencia del modelo y devuelve el arreglo de 33 landmarks. Simultáneamente, DrawingUtils dibuja el esqueleto (conectores en verde, landmarks en rojo) sobre el canvas superpuesto al video.

Los landmarks se pasan inmediatamente al tracker del ejercicio activo, que calcula los ángulos articulares relevantes y determina la fase del movimiento, el conteo de repeticiones y el nivel de retroalimentación.

## 5.3 Cálculo de ángulos articulares

Para cada articulación de interés se identifican tres landmarks: el punto proximal (A), el vértice de la articulación (B) y el punto distal (C). El ángulo en B se calcula mediante la función `calculateAngle` implementada en `src/geometry/angles.ts`:

$$\text{ángulo} = | \text{atan2}(Cy - By, Cx - Bx) - \text{atan2}(Ay - By, Ax - Bx) | \times (180 / \pi)$$

$$\text{si } \text{ángulo} > 180^\circ \rightarrow \text{ángulo} = 360^\circ - \text{ángulo}$$

El uso de `Math.atan2` (en lugar de la ley de cosenos) evita divisiones por cero cuando dos puntos son colineales y maneja correctamente todos los cuadrantes del plano. El rango de salida es siempre  $0^\circ$ – $180^\circ$ .

## 5.4 Máquinas de estados y conteo de repeticiones

Cada ejercicio implementa una máquina de estados determinista que modela las fases del movimiento. La transición entre fases utiliza histéresis de umbral doble: existen umbrales distintos para entrar y para salir de cada fase, creando una zona muerta que absorbe el ruido de los landmarks y elimina la oscilación (flutter) sin introducir latencia artificial.

El conteo de repeticiones no ocurre únicamente en la transición de fase, sino que requiere la confirmación previa de que el movimiento alcanzó su punto extremo (máximo o mínimo de ángulo, según el ejercicio). Este gate de confirmación se implementa mediante un contador de fotogramas consecutivos con tendencia sostenida, descartando spikes de ruido que no representen movimiento intencional.

## 5.5 Sistema de retroalimentación

El sistema ofrece dos canales simultáneos de retroalimentación:

**Visual:** Una barra fija en la parte inferior de la pantalla muestra el mensaje de feedback y el contador de repeticiones. Un borde izquierdo colorido codifica semánticamente el nivel de calidad: verde (buena forma), amarillo (forma mejorable) y rojo (posición de riesgo). La opacidad del fondo (80 %) y el filtro de desenfoque (backdrop-filter: blur) garantizan legibilidad sobre cualquier fondo de video.

**Auditivo:** La API nativa SpeechSynthesis del navegador emite instrucciones en español (es-ES) en los momentos clave del movimiento: al detectar el punto extremo del recorrido y al completar una repetición. El sistema combina ambos mensajes en una única locución para evitar colisiones de audio. El primer uso de la API de voz requiere un gesto del usuario (el botón "Comenzar a entrenar" del onboarding actúa como ese gesto de desbloqueo, cumpliendo el requisito de iOS Safari).

## 6. Ejercicios implementados

### 6.1 Sentadilla (Squat)

Landmarks utilizados: Caderas (23, 24), rodillas (25, 26) y tobillos (27, 28). Ángulo primario: Promedio del ángulo de ambas rodillas (cadera–rodilla–tobillo).

| Umbral           | Valor | Significado                                    |
|------------------|-------|------------------------------------------------|
| STANDING_ANGLE   | 160°  | Pierna extendida → fase "de pie"               |
| BOTTOM_ANGLE     | 100°  | Rodilla muy flexionada → fase "abajo"          |
| GOOD_DEPTH_ANGLE | 90°   | Profundidad óptima (muslos paralelos al suelo) |

**Conteo:** La repetición se contabiliza en la transición abajo → de pie, únicamente si previamente se confirmó el fondo real del movimiento (inversión de tendencia de más de 2° sostenida un fotograma).

**Retroalimentación:**

- Ángulo de rodilla  $\leq 90^\circ$ : "¡Excelente profundidad!" (verde)
- $90^\circ$ – $100^\circ$ : "Baja un poco más" (amarillo)
- Posición de pie: "Listo — baja para la sentadilla" (idle)

## 6.2 Curl de bíceps (Bicep Curl)

Landmarks utilizados: Hombros (11, 12), codos (13, 14) y muñecas (15, 16). Detección automática de vista: Si la diferencia de visibilidad entre ambos brazos supera 0.35, la aplicación infiere vista lateral y trabaja únicamente con el brazo más visible; de lo contrario, procesa ambos brazos en paralelo.

| Umbral          | Valor | Significado                            |
|-----------------|-------|----------------------------------------|
| EXTENDED_ANGLE  | 160°  | Brazo extendido → fase "extendido"     |
| FLEXED_ANGLE    | 60°   | Codo muy flexionado → fase "contraído" |
| GOOD_FORM_ANGLE | 50°   | Contracción completa                   |

**Arquitectura interna:** La clase ArmTracker encapsula la lógica de un solo brazo. BicepCurlTracker instancia dos ArmTracker y unifica el conteo con lógica OR y un cooldown de 15 fotogramas (~250 ms a 60 fps) para manejar correctamente curls bilaterales (barra o mancuernas simultáneas) y alternos (mancuernas alternas).

**Conteo:** Una repetición se registra cuando cualquiera de los dos brazos completa su ciclo extendido → contraído → extendido, con la confirmación previa de que el ángulo mínimo fue alcanzado (cima confirmada por 3 fotogramas consecutivos de tendencia ascendente).

## 6.3 Press de hombro (Shoulder Press)

Landmarks utilizados: Idénticos al curl de bíceps (hombros, codos, muñecas). Polaridad invertida: A diferencia del curl —donde el esfuerzo reduce el ángulo del codo— en el press el esfuerzo lo aumenta (las pesas suben overhead y el codo se extiende). El ángulo primario es  $\text{Math.max}(\text{ángulo\_izquierdo}, \text{ángulo\_derecho})$ , capturando el brazo más extendido como indicador de calidad.

| Umbral             | Valor | Justificación                                                     |
|--------------------|-------|-------------------------------------------------------------------|
| PRESSED_ANGLE      | 150°  | Pesas overhead → fase "presionado"                                |
| LOWERED_ANGLE      | 100°  | Pesas a nivel de hombro → fase "abajo"                            |
| GOOD_LOCKOUT_ANGLE | 145°  | Extensión completa sin hiperextensión                             |
| SAFE_LOW_ANGLE     | 80°   | Límite clínico — evitar síndrome de impingement del supraespinoso |

El límite de `SAFE_LOW_ANGLE = 80°` está fundamentado en la biomecánica del hombro: bajar los codos por debajo de la línea del hombro con carga externa comprime el tendón supraespinoso entre el acromion y la cabeza humeral. La retroalimentación roja alerta al usuario antes de que el movimiento alcance el rango de riesgo.

## 7. Stack tecnológico

| Capa                     | Tecnología                                | Versión       |
|--------------------------|-------------------------------------------|---------------|
| Detección de poses       | MediaPipe Pose Landmarker (variante Lite) | 0.10.35       |
| WASM runtime             | jsDelivr CDN                              | @0.10.35/wasm |
| Framework UI             | React                                     | 19.2.5        |
| Lenguaje                 | TypeScript                                | 6.0.2         |
| Build tool               | Vite                                      | 8.0.10        |
| Cámara                   | getUserMedia (API nativa del navegador)   | —             |
| Renderizado de esqueleto | HTML5 Canvas API + DrawingUtils           | —             |
| Feedback de voz          | Web Speech API (SpeechSynthesis)          | —             |
| PWA                      | Service Worker manual + Web App Manifest  | —             |
| Deploy                   | Vercel (plan Hobby gratuito)              | —             |
| HTTPS local              | @vitejs/plugin-basic-ssl                  | 2.3.0         |

La decisión de no utilizar librerías adicionales de gestión de estado (Redux, Zustand) ni de animación (Framer Motion) fue deliberada: el estado de la aplicación es lo suficientemente simple como para manejarse con los hooks nativos de React (useState, useRef, useEffect), y cualquier dependencia adicional aumenta el tamaño del bundle que el usuario debe descargar en su primera visita.

## 8. Decisiones de diseño relevantes

### 8.1 Decisión de servidor e infraestructura

Todo el procesamiento ocurre en el dispositivo del usuario. Esto elimina los costos de infraestructura, protege la privacidad del usuario (el video de la cámara nunca sale del dispositivo) y hace que la aplicación funcione sin conexión una vez que los recursos están cacheados.

### 8.2 Service Worker con estrategia diferenciada

El Service Worker distingue entre dos tipos de recursos:

**HTML de navegación (index.html):** Estrategia network-first. La aplicación siempre intenta obtener la versión más reciente; el caché actúa solo como fallback offline. Esto garantiza que los usuarios con la PWA instalada reciban actualizaciones inmediatamente.

**Assets estáticos (JavaScript, CSS, íconos):** Estrategia cache-first. Vite genera nombres de archivo con hash de contenido, por lo que cada URL es inmutable; servirlos desde el caché es seguro y elimina solicitudes de red innecesarias.

### 8.3 Onboarding de cuatro pantallas

La aplicación presenta al usuario un flujo de bienvenida en su primera apertura: pantalla de splash, explicación del funcionamiento, solicitud de permisos de cámara y configuración inicial (cámara frontal o trasera). El flujo resuelve el problema de que el navegador solicite permisos de cámara sin contexto, lo que frecuentemente resulta en que el usuario los deniega por desconfianza.

### 8.4 Delay de hardware al cambiar de cámara

En modo PWA instalada (standalone), el cambio entre cámara frontal y trasera produce una colisión de hardware: `track.stop()` es síncrono en JavaScript, pero el sensor físico de la cámara no libera el recurso inmediatamente. Se implementó un delay de 450 ms detectado mediante un flag de referencia en el ciclo de vida del efecto de React, valor determinado empíricamente para cubrir la mayoría de los dispositivos Android e iOS.

## 9. Resultados

La aplicación fue probada en dispositivos Android con Chrome y en iOS con Safari. Los resultados observados durante las pruebas son los siguientes:

**Latencia de detección:** Subjetivamente imperceptible en dispositivos de gama media (2022 en adelante). La variante Lite del modelo procesa cada fotograma en menos de 30 ms en la mayoría de los casos, manteniendo la cadencia de 30 fps del stream de cámara.

**Precisión de conteo:** El gate de confirmación por fotogramas consecutivos eliminó las repeticiones falsas detectadas en versiones tempranas por movimientos bruscos de la cámara o por el ruido inherente de los landmarks en movimientos rápidos.

**Usabilidad:** El onboarding redujo la tasa de denegación de permisos de cámara al presentar el contexto antes del diálogo nativo del navegador. La retroalimentación de voz fue valorada positivamente durante las pruebas por permitir al usuario mantener la vista en el espejo de la pantalla sin necesidad de leer el texto del overlay.

**PWA:** La aplicación se instala correctamente en Android (Chrome) e iOS (Safari → "Agregar a pantalla de inicio") y se comporta como una aplicación nativa en modo standalone.

### 9.1 Link para visualizar resultados

Link de YouTube para demo y demostración de la aplicación: <https://youtu.be/SCQXUJRCDiE>

## 10. Limitaciones conocidas

1. Dependencia de la posición de la cámara: La calidad de la detección depende de que el cuerpo completo sea visible y bien iluminado. En condiciones de poca luz o con oclusiones parciales (ropa holgada, accesorios), la precisión de los landmarks disminuye.
2. Calibración por usuario: Los umbrales angulares fueron determinados empíricamente para una persona de proporciones corporales promedio. Usuarios con proporciones muy diferentes (brazos muy largos, torso muy corto) podrían requerir ajustes.
3. Vista de la cámara: La sentadilla y el press de hombro ofrecen mejor cobertura con vista lateral o de 45°. La detección automática de vista en el curl de bíceps es heurística y puede presentar errores en configuraciones inusuales de encuadre.
4. iOS Safari — SpeechSynthesis: El primer disparo de voz requiere un gesto previo del usuario (resuelto con el botón del onboarding). En iOS, las voces en español pueden variar según la región configurada en el dispositivo.

## 11. Conclusiones

El proyecto demostró que es técnicamente viable construir una aplicación de entrenamiento personal con retroalimentación en tiempo real usando exclusivamente tecnologías web gratuitas y el procesador del propio dispositivo móvil. La integración de MediaPipe Pose Landmarker como motor de inferencia permitió que el equipo se concentrara en la lógica de dominio —biomecánica de los ejercicios, diseño de las máquinas de estados, calidad del feedback— sin necesidad de entrenar ni mantener un modelo propio.

Desde el punto de vista de la ingeniería de software, el proyecto consolidó prácticas de diseño orientadas a la robustez en condiciones adversas: histéresis para señales ruidosas, confirmación por fotogramas consecutivos para eventos que deben ser intencionales, estrategias diferenciadas de caché para recursos con distintos ciclos de vida, y manejo defensivo de APIs del navegador que pueden fallar silenciosamente.

El trabajo realizado constituye una base sólida sobre la que podrían construirse extensiones aspiracionales: un clasificador automático de ejercicio entrenado con las secuencias de los 33 keypoints, retroalimentación personalizada basada en el historial del usuario, o soporte para ejercicios adicionales como planchas y lunges.

## 12. Referencias

- Google LLC. (2024). MediaPipe Pose Landmarker guide. Google for Developers. [https://developers.google.com/mediapipe/solutions/vision/pose\\_landmarker](https://developers.google.com/mediapipe/solutions/vision/pose_landmarker)
- Google LLC. (2024). MediaPipe Tasks Vision — JavaScript API reference. <https://developers.google.com/mediapipe/api/solutions/js/tasks-vision>
- Mozilla Developer Network. (2024). MediaDevices.getUserMedia(). MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>
- Mozilla Developer Network. (2024). Progressive web apps (PWAs). MDN Web Docs. [https://developer.mozilla.org/en-US/docs/Web/Progressive\\_web\\_apps](https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps)
- Mozilla Developer Network. (2024). SpeechSynthesis. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/API/SpeechSynthesis>
- React Team. (2024). React 19 documentation. <https://react.dev>
- Vitejs. (2024). Vite — Next Generation Frontend Tooling. <https://vitejs.dev>
- Newell, A., Yang, K., & Deng, J. (2016). Stacked hourglass networks for human pose estimation. European Conference on Computer Vision (ECCV). Springer, Cham.
- Cao, Z., Simon, T., Wei, S. E., & Sheikh, Y. (2017). Realtime multi-person 2D pose estimation using part affinity fields. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).